CSS 262: Linux Administration

Firewalls & Packet Filtering

Lecture 9: Perimeter Defense with iptables & UFW

📋 Today's Agenda

Part 1: Firewall Fundamentals

- What is a firewall & why it matters
- Packet filtering vs. stateful inspection
- Netfilter framework & Linux firewall stack

Part 2: iptables Deep Dive

- Chains: INPUT, FORWARD, OUTPUT
- Rule syntax & default policies
- Stateful inspection (conntrack)

Part 3: UFW – Uncomplicated Firewall

- UFW as iptables frontend
- allow/deny rules & application profiles
- Common patterns & best practices

Part 4: Advanced & Real-World

- Persisting rules, logging
- Combining with SSH hardening
- nftables & firewalld overview

🔄 Quick Recap: Week 8

SSH Hardening

- Key-based auth (Ed25519), disable passwords
- `PermitRootLogin no`, `AllowUsers`
- Fail2Ban for brute-force protection
- `~/.ssh/config`, port forwarding

Key Takeaways

- SSH is the #1 attack target on Linux servers
- Defense in depth: keys + Fail2Ban + non-default port
- **Today**: Add another layer – **firewall rules**
- Firewall controls *which* traffic reaches SSH at all

🛡️ **Perimeter Defense:** Firewalls filter traffic before it reaches services. SSH hardening + firewall = layered security.

Part 1

🛡️ Firewall Fundamentals

🛡️ What is a Firewall?

Definition

- **Firewall**: Network security device that monitors and controls traffic
- Operates at Layer 3 (IP) and Layer 4 (TCP/UDP)
- Decides: allow or drop packets based on rules
- Sits between trusted (internal) and untrusted (external) networks

Why Firewalls Matter

- **Defense in depth** – even if a service has bugs, firewall can block access
- **Least privilege** – only allow what's needed
- **Reduce attack surface** – close unused ports
- **Logging** – detect scan/attack attempts

⚠️ **Reality:** Default Linux install often has no firewall enabled. Exposed services = easy targets for automated scanners.

📦 Packet Filtering vs. Stateful Inspection

Stateless (Packet Filtering)

- Each packet evaluated **independently**
- Rules: source IP, dest IP, port, protocol
- **Problem**: Can't distinguish reply from new connection
- Example: Allow port 22 in → must also allow port 22 out for replies

Stateful (Connection Tracking)

- Tracks **connections** (conntrack)
- Knows: "this packet is part of established session"
- **Benefit**: Allow established/related, drop new unsolicited
- Example: Allow SSH out → replies automatically allowed

✅ **Linux uses stateful inspection** via `nf_conntrack` — essential for practical firewall rules.

🏗️ Netfilter & the Linux Firewall Stack

Netfilter Framework

- **Netfilter**: Kernel subsystem for packet manipulation
- **iptables**: User-space tool to configure Netfilter rules
- **nftables**: Modern successor (we'll touch on it later)

Packet Flow Through Chains

```
[Internet] → PREROUTING → FORWARD → POSTROUTING → [Internal]
                 ↓
             INPUT → [Local Process]
                 ↑
             OUTPUT ← [Local Process]
```

🔑 **Three main chains:** `INPUT` (to this host), `OUTPUT` (from this host), `FORWARD` (routed traffic).

📊 Netfilter Tables

Table	Purpose	Common Use
filter	Default – allow/drop	Most firewall rules
nat	NAT, port forwarding	Masquerading, DNAT
mangle	Packet modification	TOS, TTL
raw	Bypass conntrack	Performance tuning

💡 **Default:** `iptables -L` shows the `filter` table. Use `-t nat` for NAT rules.

⚠️ **Order matters:** Rules are evaluated top-to-bottom. First match wins.

Part 2

🔧 iptables Deep Dive

🔗 The Three Main Chains

INPUT

Traffic **destined for this host**

- Incoming SSH, HTTP, etc.
- **Most rules go here** for a server
- Block unwanted external access

OUTPUT

Traffic **originating from this host**

- Outgoing connections
- Often permissive (allow all)
- Restrict for compliance/egress control

FORWARD

Traffic **passing through** (routing)

- Used when host acts as router/gateway
- For simple servers: often empty

📝 iptables Rule Syntax

Basic Structure

```
iptables -A CHAIN -p PROTOCOL --dport PORT -j TARGET
```

Option	Meaning	Example
`-A`	Append rule to chain	`-A INPUT`
`-p`	Protocol	`-p tcp`, `-p udp`, `-p icmp`
`--dport`	Destination port	`--dport 22`
`-s`	Source IP	`-s 192.168.1.0/24`
`-j`	Jump to target	`-j ACCEPT`, `-j DROP`, `-j REJECT`

🚨 **Note:** `--dport` and `--sport` require `-p tcp` or `-p udp`.

🎯 iptables Targets

Target	Action
ACCEPT	Allow the packet
DROP	Silently discard (no response)
REJECT	Discard + send ICMP/port unreachable
LOG	Log and continue to next rule
RETURN	Return to calling chain

⚠️ **DROP vs REJECT:** DROP = stealth (attacker gets no response). REJECT = polite (client gets error). For security, DROP is often preferred on INPUT.

✅ **Default policy:** What happens if no rule matches. Set with `iptables -P CHAIN ACCEPT|DROP`.

📋 iptables: Common Commands

```
# List rules (filter table)
iptables -L -n -v

# Set default policy
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

# Allow established/related (stateful!)
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT

# Allow SSH
iptables -A INPUT -p tcp --dport 22 -j ACCEPT

# Allow loopback
iptables -A INPUT -i lo -j ACCEPT
```

🚨 **Order:** Established/related FIRST, then specific allows, default policy last. Wrong order = lockout!

🔄 Stateful Inspection: conntrack

Connection States

- **NEW**: First packet of new connection
- **ESTABLISHED**: Connection in progress
- **RELATED**: Related (e.g. FTP data)
- **INVALID**: Malformed or suspicious

The Golden Rule

```
iptables -A INPUT -m conntrack \
  --ctstate ESTABLISHED,RELATED -j ACCEPT
```

- Allows **replies** to outbound connections
- Without this: SSH out works, but responses get dropped!
- Place this rule **early** in the chain

🌐 Example: Minimal Server Ruleset

```
iptables -F && iptables -X
iptables -P INPUT DROP
iptables -P FORWARD DROP
iptables -P OUTPUT ACCEPT

iptables -A INPUT -i lo -j ACCEPT
iptables -A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -p tcp --dport 80 -j ACCEPT
iptables -A INPUT -p tcp --dport 443 -j ACCEPT
iptables -A INPUT -p icmp --icmp-type echo-request -j ACCEPT
```

📝 iptables Logging

```
# Log dropped packets (before DROP rule)
iptables -A INPUT -m limit --limit 5/min -j LOG \
  --log-prefix "iptables-INPUT-drop: " --log-level 4

# View logs
sudo dmesg | grep iptables
# Or: journalctl -k | grep iptables
```

💡 **--limit** prevents log flooding. Without it, a port scan could fill disk with log entries.

⚠️ **LOG target:** Packet continues to next rule after logging. Put LOG before DROP.

💾 Persisting iptables Rules

Rules are in-memory – lost on reboot!

```
# Save rules (Debian/Ubuntu)
sudo iptables-save > /etc/iptables/rules.v4

# Restore on boot: install iptables-persistent
sudo apt install iptables-persistent
# Saves to /etc/iptables/rules.v4 on install/reboot

# Manual save/restore
sudo iptables-save > rules.backup
sudo iptables-restore rules.backup
```

✅ **Debian/Ubuntu:** `iptables-persistent` package

💡 **RHEL/Fedora:** `firewalld` or `iptables-service`

Part 3

🔥 UFW – Uncomplicated Firewall

🔥 What is UFW?

Uncomplicated Firewall

- **Frontend** for iptables
- Default on Ubuntu/Debian
- Simpler syntax than raw iptables
- Still generates iptables rules under the hood

Why Use UFW?

- Easier to learn and remember
- Application profiles (predefined rules)
- Less error-prone
- Good for most server use cases

💡 **UFW translates to iptables.** Run `iptables -L` after UFW rules to see the result.

🔧 UFW Basic Commands

```
# Enable/disable (asks for confirmation)
sudo ufw enable
sudo ufw disable

# Default policies
sudo ufw default deny incoming
sudo ufw default allow outgoing

# Allow/deny rules
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw deny 23/tcp

# By application name
sudo ufw allow ssh
sudo ufw allow 'Nginx Full'
```

💡 Application profiles in `/etc/ufw/applications.d/`. List with `ufw app list`.

📋 UFW Rule Syntax

```
# Allow from specific IP
sudo ufw allow from 192.168.1.100

# Allow from subnet
sudo ufw allow from 192.168.1.0/24 to any port 22

# Deny from IP
sudo ufw deny from 10.0.0.50

# Allow to specific port with protocol
sudo ufw allow 53/udp
sudo ufw allow 53/tcp
```

✅ Rules are **stateful** by default – replies allowed automatically

💡 `ufw status numbered` – show rules with numbers for delete

📊 UFW Status & Management

```
# Status
sudo ufw status
sudo ufw status verbose
sudo ufw status numbered

# Delete rule by number
sudo ufw delete 3

# Delete by rule
sudo ufw delete allow 22/tcp

# Reset (disable + flush all rules)
sudo ufw reset

# Reload (apply changes)
sudo ufw reload
```

🚨 **ufw reset** removes everything. Use with caution. Ensure SSH is allowed before enabling!

🔒 UFW: Secure Default Setup

Recommended Order

```
# 1. Allow SSH FIRST (avoid lockout!)
sudo ufw allow 22/tcp
# Or: sudo ufw allow ssh

# 2. Set defaults
sudo ufw default deny incoming
sudo ufw default allow outgoing

# 3. Add other allows
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp

# 4. Enable (will prompt)
sudo ufw enable

# 5. Verify
sudo ufw status verbose
```

🚨 **Always allow SSH before enabling!** Locking yourself out of a remote server = need console access.

📝 UFW Logging

```
# Enable logging (off, low, medium, high, full)
sudo ufw logging on
sudo ufw logging medium

# Logs go to
# /var/log/ufw.log  (or syslog, depending on config)
```

💡 **low** = blocked only. **medium** = blocked + policy violations. **high** = all packets.

✅ Use `medium` for typical servers – see attacks without log flood

Part 4

🚀 Advanced & Real-World

🛡️ Firewall + SSH Hardening: Layered Defense

Combine Both

- Firewall: restrict which IPs can reach port 22
- SSH: key-only auth, no root, Fail2Ban
- Example: Allow SSH only from office VPN subnet

Example: Restrict SSH by IP

```
# UFW
sudo ufw deny 22/tcp
sudo ufw allow from 203.0.113.0/24 to any port 22

# iptables equivalent
iptables -A INPUT -p tcp -s 203.0.113.0/24 --dport 22
iptables -A INPUT -p tcp --dport 22 -j DROP
```

📊 Common Firewall Patterns

Pattern	Use Case
Default deny in	Block all, allow explicit – most secure
Default allow out	Let server make outbound connections
Allow loopback	Required for local services
ESTABLISHED,RELATED first	Stateful – allow replies
Log before drop	Debug and detect scans
Restrict by IP	Limit SSH/admin to trusted networks

✅ **Principle:** Least privilege. Only open what's needed. Prefer IP allowlists over "allow from any."

🔄 nftables & firewalld Overview

nftables

- **Successor** to iptables
- Single framework, better performance
- Different syntax
- `nft` command, rules in `/etc/nftables.conf`

firewalld

- Default on RHEL, Fedora, CentOS
- Zone-based (public, internal, etc.)
- Dynamic updates without restart
- Uses iptables or nftables as backend

💡 **For this course:** Focus on iptables + UFW. nftables/firewalld are good to know for RHEL environments.

✅ Firewall Security Checklist

Configuration

- ☐ Default deny incoming
- ☐ Allow SSH before enabling
- ☐ ESTABLISHED,RELATED rule (stateful)
- ☐ Allow loopback (lo)
- ☐ Only open required ports
- ☐ Restrict SSH by IP if possible

Operations

- ☐ Persist rules (iptables-persistent / ufw)
- ☐ Enable logging (medium level)
- ☐ Test from second session before closing SSH
- ☐ Document rules for audit
- ☐ Combine with SSH hardening

📊 Summary: Firewalls & Packet Filtering

Key Concepts

1. **Netfilter/iptables**: Kernel + user-space firewall
2. **Chains**: INPUT, OUTPUT, FORWARD
3. **Stateful**: conntrack – ESTABLISHED,RELATED
4. **Targets**: ACCEPT, DROP, REJECT, LOG
5. **UFW**: Simpler frontend for iptables
6. **Persistence**: iptables-save, iptables-persistent

Rule Order Matters

1. Loopback
2. ESTABLISHED,RELATED
3. Specific allows (SSH, HTTP, etc.)
4. Log (optional)
5. Default policy (DROP)

🎯 Learning Objectives: Did We Achieve?

By now, you should be able to:

- ✅ Explain packet filtering vs. stateful inspection
- ✅ Describe the INPUT, FORWARD, OUTPUT chains
- ✅ Write iptables rules with `-A`, `-p`, `--dport`, `-j`
- ✅ Use conntrack for ESTABLISHED,RELATED
- ✅ Configure UFW with allow/deny rules
- ✅ Persist firewall rules across reboots
- ✅ Combine firewall with SSH hardening
- ✅ Enable logging and interpret firewall logs

🎓 **Next Week:** Mandatory Access Control (SELinux) – contexts, booleans, and audit2why!

🧪 Lab Practice: Firewalls

Exercise 1: iptables from Scratch

On a lab VM, flush rules and build a minimal ruleset: default deny INPUT, allow loopback, ESTABLISHED/RELATED, SSH (22), HTTP (80). Test with `curl` and `ss`.

Exercise 2: UFW Setup

Configure UFW with the same rules. Use `ufw allow ssh`, `ufw allow 80/tcp`. Enable and verify with `ufw status verbose`. Compare with `iptables -L`.

Exercise 3: Restrict SSH by IP

Add a rule to allow SSH only from your lab subnet (e.g. 192.168.1.0/24). Test from allowed and blocked IPs. Document the rule.

Exercise 4: Logging

Enable UFW logging at medium level. Trigger a blocked connection (e.g. try to connect to closed port from another machine). Inspect `/var/log/ufw.log` or `journalctl`.

🔗 Additional Resources

Documentation

- netfilter/iptables – Official project
- UFW Ubuntu Documentation – UFW guide
- nftables wiki – nftables reference

Books & Guides

- *"Linux Firewalls"* by Steve Suehring – iptables/nftables
- Arch Wiki: iptables – Comprehensive reference
- CIS Benchmark – Firewall hardening

Practice

- Set up two VMs: one as "server" with firewall, one as "attacker" – run `nmap` scans
- Build a ruleset that allows only HTTP/HTTPS and SSH from specific IP

Questions?

Next: Mandatory Access Control (SELinux)